总体对比先说清

核心区别：

- **Vue2**：基于 `Object.defineProperty` 劫持对象属性的 `getter/setter` ，配合 `Dep / Watcher` 做依赖收集和派发更新。
- **Vue3**：基于 `Proxy` 对整个对象做“代理”，内部用 `track / trigger` 和 `effect` 系统实现更通用的响应式。

下面分开讲，两者怎么实现“数据变了 → 视图更新”。

Vue2 的响应式原理

1. 核心技术： `Object.defineProperty`

- 初始化数据时（ `new Vue({ data(){ return {...} } })` ）：
  - Vue 会遍历 `data` 中每个属性，对每个属性调用 `Object.defineProperty` ：
    - 自定义 `get` ：读取时做“依赖收集”
    - 自定义 `set` ：修改时做“派发更新”

```
Object.defineProperty(obj, 'foo', {
  get() { /* 依赖收集 */ },
  set(newVal) { /* 通知相关 watcher 更新 */ }
})

```

- 对象要先“被观察”（ `observe` ），属性才能变成响应式。

2. 依赖收集： `Dep` 和 `Watcher`

- **Watcher**：可以理解为“订阅者”，代表某个依赖数据的副作用逻辑：
  - 渲染 `render watcher` （模板 / 组件渲染）
  - `computed watcher`
  - 用户自定义 `watch`
- **Dep**：每个被劫持的属性都有一个 `Dep` 实例（依赖桶）：
  - `get` 时，如果当前有正在评估的 `Watcher` ，就把这个 `Watcher` 加入 `Dep` 。
  - `set` 时，通知 `Dep` 中所有 `Watcher` 重新执行（重新渲染、重新计算等）。

流程简化为：

- 模板渲染时 → 访问响应式数据 → 触发 `get` → 当前 `Watcher` 记录到属性的 `Dep` 里。
- 数据修改时 → 触发 `set` → `Dep.notify()` → 对应 `Watcher.update()` → 组件重新渲染。

3. 数组的特殊处理

`Object.defineProperty` 无法直接拦截数组的索引操作，所以 Vue2：

- 通过**重写部分数组变更方法**（`push/pop/shift/unshift/splice/sort/reverse`）：
  - 这些方法被包一层，内部先执行原始逻辑，再触发依赖更新。
- 对数组的**索引赋值、修改** `length` 无法被监听到，需要：
  - `Vue.set(arr, index, value)`
  - 或者 `this.arr.splice(index, 1, value)`

4. Vue2 的局限

- **新增/删除属性无法被监听**：必须用 `Vue.set` / `Vue.delete` 。
- **数组下标赋值、length 变化不会触发更新。**
- 深层对象需要递归 `defineProperty` ，初始化成本较高。
- 无法直接监听 `Map/Set` 等新数据结构。

Vue3 的响应式原理

1. 核心技术：`Proxy` + `Reflect`

- Vue3 使用 `Proxy` 直接代理整个对象：

```
const state = new Proxy(target, {
  get(target, key, receiver) {
    // track：依赖收集
    return Reflect.get(target, key, receiver)
  },
  set(target, key, value, receiver) {
    // trigger：派发更新
    const res = Reflect.set(target, key, value, receiver)
    return res
  },
  // 还可以拦截 deleteProperty, has, ownKeys 等
})

```

- 优点：
  - 可以拦截“新增属性、删除属性、数组索引、 `length` 修改”等几乎所有操作。
  - 不需要一开始就递归遍历所有属性，访问到时再“懒代理”（按需递归）。

2. `effect / track / trigger` 机制

Vue3 把响应式系统抽象为三个核心概念：

- `effect(fn)`：注册一个副作用函数（类似 Vue2 的 `Watcher`）：
  - 执行 `fn` 时，内部读取响应式数据会触发 `track`。
- `track(target, key)`：依赖收集：
  - 把“当前正在执行的 effect”记录到对应 `target + key` 的依赖集合中。
- `trigger(target, key)`：派发更新：
  - 取出依赖于这个 `target + key` 的所有 effect，重新执行。

典型实现结构：

- 使用 `WeakMap` 存所有依赖关系：
  - `targetMap: WeakMap<target, Map<key, Set<effect>>>`

访问流程：

- 读属性（`get`）→ `track(target, key)` → 收集依赖。
- 写属性（`set`）→ `trigger(target, key)` → 触发更新。

3. 多种响应式 API 的本质

- `reactive(obj)`：给整个对象创建 `Proxy`。
- `ref(value)`：通过 `value` 属性实现响应式（内部仍会用 `effect/track/trigger`）。
- `readonly` / `shallowReactive` 等：在同一套 `Proxy` trap 基础上做变体。

本质上都是围绕 `effect + track + trigger` 这套系统在包装不同的使用方式。

4. Vue3 相对 Vue2 的优势

- **能监听属性的新增/删除**：无需 `Vue.set` / `Vue.delete`。
- **能正确监听数组下标、`length`、`Map/Set` 操作等。**
- **初始化成本更低**：不需要递归 `defineProperty`，按访问懒代理。
- 响应式系统抽象更清晰，可独立为 `@vue/reactivity` 使用（比如组合式 API、跨平台框架）。

总结对比记忆法

- 实现基础
  - **Vue2**：`Object.defineProperty` + `Dep/Watcher`。
  - **Vue3**：`Proxy` + `Reflect` + `effect/track/trigger`。
- 监听能力
  - **Vue2**：属性新增删除、数组下标、`length` 需要额外 API 或有缺陷。
  - **Vue3**：几乎所有操作都可拦截（包括 Map / Set）。
- 设计层面
  - **Vue2**：以组件渲染 Watcher 为核心，响应式系统和 Vue 耦合较重。
  - **Vue3**：响应式独立为通用库，更通用、更可复用。